Running the Nextcloud + Euro-Office Stack on Kubernetes

Jan

August 5, 2026

Running the Nextcloud + Euro-Office stack on Kubernetes

This document translates the local **Podman** installation of Nextcloud with Euro-Office (recorded in the other documents of this directory) into a **Kubernetes** deployment. It provides the complete Kubernetes manifests and the step-by-step procedure to run the same workload - Nextcloud, PostgreSQL, Redis and the Euro-Office document server - in a Kubernetes cluster, including all the fixes that were necessary under Podman and how they are handled in Kubernetes.

> [!WARNING] **Artificial-intelligence notice.** This document was created with the assistance of **artificial intelligence** on the basis of the recorded Podman installation in this directory. It has **not** been validated against a real Kubernetes cluster. The manifests and commands are provided as a starting point; a real deployment on Kubernetes will almost certainly take **considerably more time** than the estimate given in section 12. Plan for rework, validation and troubleshooting time on your actual cluster.

Document	Content
overview.md	Overview of the local (air-gapped) Podman installation
installation-steps.md	Full AIO install log under Podman
local-nextcloud-fixes.md	The Podman / local-only fixes in detail
nextcloud-euro-office-integration.md	Euro-Office integration and the document-saving fix
euro-office-github.md	Euro-Office upstream install guide (GitHub-based)
final-install.md	The concrete Podman containers and credentials used

1. Goal

The local setup runs three Podman containers on a single host (see final-install.md):

Podman container	Image	Role
`nextcloud`	`quay.io/linuxserver.io/nextcloud:latest`	Nextcloud web application (nginx + PHP)
`nextcloud-db`	`quay.io/fedora/postgresql-16:latest`	PostgreSQL database
`euro-office`	`ghcr.io/euro-office/documentserver:latest`	Euro-Office document server

plus (in the AIO variant of overview.md) the AIO orchestration layer (`nextcloud-aio-mastercontainer`, `nextcloud-aio-apache`, `nextcloud-aio-redis`) and the AIO-managed Euro-Office container `nextcloud-aio-eurooffice`.

The goal here is to run the **same workload** in a Kubernetes cluster: a Nextcloud instance reachable through a stable hostname, backed by PostgreSQL and Redis, with the Euro-Office document server integrated so that documents can be opened **and saved**.

2. What changes compared to the Podman installation

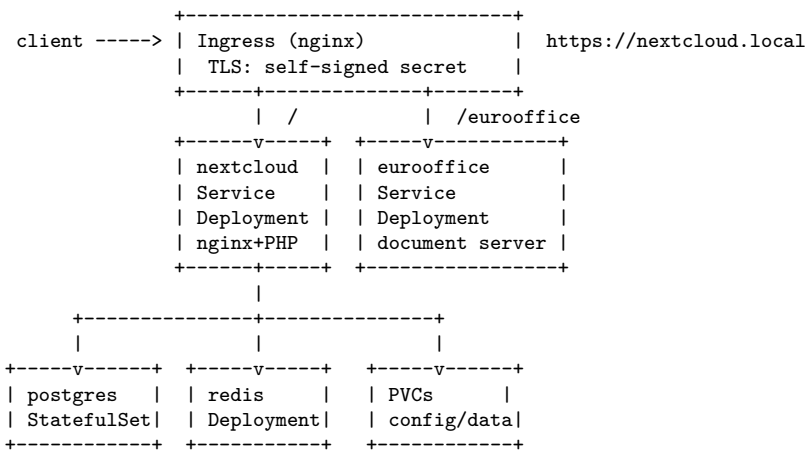
Nextcloud All-in-One (AIO) **cannot be run as-is in Kubernetes**. AIO is a Docker/Podman orchestrator: its mastercontainer talks to the container engine socket and creates sibling containers on a private bridge network. Kubernetes already *is* an orchestrator, so that role is simply replaced:

Podman / AIO concept	Kubernetes replacement
AIO mastercontainer (creates containers via the Docker socket)	`kubectl` / `kubectl apply` + the Kubernetes API
Container definitions patched in `containers.json`	Plain YAML manifests (the fixes become *manifest fields*, no runtime patching)
`nextcloud-aio-apache` reverse proxy + Caddy TLS	Ingress controller + TLS secret
Private `nextcloud-aio` bridge network with Docker-DNS names	Kubernetes Services (`nextcloud-postgres`, `nextcloud-redis`, ...)
`--restart always` / systemd user units	Deployment / StatefulSet replicas + the cluster scheduler
Volumes `nextcloud_aio_mastercontainer`, named volumes	PersistentVolumeClaims

Everything that had to be *fixed at runtime* under Podman (binding port 443, self-signed TLS, php-fpm `listen.allowed_clients`, Euro-Office `local.json` environment variables) is expressed directly in the manifests in Kubernetes. Section 9 walks through each fix.

3. Architecture overview

Deployed into the namespace `nextcloud`:



The browser reaches everything through a single Ingress on the hostname `nextcloud.local`:

- `https://nextcloud.local/` → Nextcloud
- `https://nextcloud.local/eurooffice` → Euro-Office document server (used as `DocumentServerUrl`)

4. Prerequisites

1. **A Kubernetes cluster** - any conformant cluster (kubeadm, k3s, RKE2, managed). For an air-gapped deployment (as in the Podman setup) the images must already be available in a local registry that the cluster can pull from; the manifests below reference the images from the Podman installation.
2. **kubectl** configured for the cluster.
3. **An Ingress controller** that supports TLS termination. This document assumes **ingress-nginx** (`kubectl get ingressclass nginx` should return a class).
4. **A default StorageClass** that provides `ReadWriteOnce` volumes (`kubectl get storageclass`).

5. **DNS or hosts entries** so that `nextcloud.local` resolves to the ingress controller address on every client, and so that pods inside the cluster can resolve it too (see section 9, fix 4).

5. Kubernetes manifests

The complete manifests, in the order in which they are applied. Each file shows the manifest to save (e.g. as `kubernetes/<name>.yaml` in the current working directory). The default values match the Podman installation (see final-install.md); change the secrets before a real deployment.

5.1 namespace.yaml

```
apiVersion: v1
kind: Namespace
metadata:
  name: nextcloud
```

5.2 secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: nextcloud-secrets
  namespace: nextcloud
type: Opaque
stringData:
  DB_NAME: nextcloud
  DB_USER: nextcloud
  DB_PASSWORD: nextcloud-secret
  JWT_SECRET: euro-office-jwt-secret-at-least-32-chars
  REDIS_PASSWORD: nextcloud-redis-secret
```

> [!NOTE] `stringData` writes the values in plain text into the Secret. For a real deployment generate strong random values (`openssl rand -base64 32`), or better: create the Secret from the command line with `kubectl create secret generic nextcloud-secrets --namespace nextcloud --from-literal=...` so the values never land in a file. The JWT secret **must** be at least 32 characters (HS256).

5.3 postgres.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: postgres-data
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 20Gi
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: nextcloud-postgres
  namespace: nextcloud
spec:
  serviceName: nextcloud-postgres
  replicas: 1
  selector:
    matchLabels:
      app: nextcloud-postgres
  template:
    metadata:
      labels:
        app: nextcloud-postgres
    spec:
```

```
      containers:
        - name: postgres
          image: docker.io/library/postgres:16
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: nextcloud-secrets
                  key: DB_USER
            - name: POSTGRES_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: nextcloud-secrets
                  key: DB_PASSWORD
            - name: POSTGRES_DB
              valueFrom:
                secretKeyRef:
                  name: nextcloud-secrets
                  key: DB_NAME
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
          readinessProbe:
            exec:
              command: ["pg_isready", "-U", "$(POSTGRES_USER)"]
            initialDelaySeconds: 10
            periodSeconds: 10
          livenessProbe:
            exec:
              command: ["pg_isready", "-U", "$(POSTGRES_USER)"]
            initialDelaySeconds: 30
            periodSeconds: 30
          resources:
            requests:
              memory: 256Mi
              cpu: 250m
            limits:
              memory: 2Gi
              cpu: "2"
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 20Gi
---
apiVersion: v1
kind: Service
metadata:
  name: nextcloud-postgres
  namespace: nextcloud
spec:
  selector:
    app: nextcloud-postgres
  ports:
    - port: 5432
      targetPort: 5432
```

[!NOTE] The Podman installation used `quay.io/fedora/postgresql-16:latest` (with `POSTGRESQL_USER/PASSWORD/DATABASE` variables and data directory `/var/lib/pgsql/data`). That image is discontinued upstream; the manifest above uses the official `docker.io/library/postgres:16`. If your air-gapped registry only contains the Fedora image, use it instead and adjust the environment variable names and `mountPath` accordingly.

5.4 redis.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextcloud-redis
  namespace: nextcloud
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nextcloud-redis
  template:
    metadata:
      labels:
        app: nextcloud-redis
    spec:
      containers:
        - name: redis
          image: docker.io/library/redis:7-alpine
          command:
            - redis-server
            - --requirepass
            - "$(REDIS_PASSWORD)"
            - --appendonly
            - "yes"
          env:
            - name: REDIS_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: nextcloud-secrets
                  key: REDIS_PASSWORD
          ports:
            - containerPort: 6379
          readinessProbe:
            exec:
              command: ["redis-cli", "-a", "$(REDIS_PASSWORD)", "ping"]
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            exec:
              command: ["redis-cli", "-a", "$(REDIS_PASSWORD)", "ping"]
            initialDelaySeconds: 15
            periodSeconds: 30
          resources:
            requests:
              memory: 64Mi
              cpu: 50m
            limits:
              memory: 512Mi
              cpu: "1"
      volumes:
        - name: data
          emptyDir: {}
---
apiVersion: v1
kind: Service
metadata:
  name: nextcloud-redis
  namespace: nextcloud
spec:
  selector:
    app: nextcloud-redis
  ports:
    - port: 6379
      targetPort: 6379
```

[!NOTE] Redis is a cache: an `emptyDir` volume is sufficient. If you want the cache to survive node restarts, replace it with a `PersistentVolumeClaim`.

5.5 nextcloud.yaml

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nextcloud-config
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nextcloud-data
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 100Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nextcloud
  namespace: nextcloud
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nextcloud
  template:
    metadata:
      labels:
        app: nextcloud
    spec:
      # Make mounted volumes writable by the image's user (uid/gid 1000, PUID/PGID)
      securityContext:
        fsGroup: 1000
      # Let the nextcloud pod resolve the public hostname to the ingress address
      # (the Euro-Office callback and Nextcloud-internal requests need this too).
      hostAliases:
        - ip: "192.168.0.114"   # CHANGE ME: ingress controller address
          hostnames:
            - "nextcloud.local"
      containers:
        - name: nextcloud
          image: quay.io/linuxserver.io/nextcloud:latest
          env:
            - name: PUID
              value: "1000"
            - name: PGID
              value: "1000"
            - name: TZ
              value: Europe/Brussels
          ports:
            - name: http
              containerPort: 80
            - name: https
              containerPort: 443
          volumeMounts:
            - name: config
              mountPath: /config
            - name: data
              mountPath: /data
          readinessProbe:
            httpGet:
```

```
            path: /status.php
            port: http
          initialDelaySeconds: 30
          periodSeconds: 10
        livenessProbe:
          httpGet:
            path: /status.php
            port: http
          initialDelaySeconds: 60
          periodSeconds: 30
        resources:
          requests:
            memory: 512Mi
            cpu: 250m
          limits:
            memory: 2Gi
            cpu: "2"
    volumes:
      - name: config
        persistentVolumeClaim:
          claimName: nextcloud-config
      - name: data
        persistentVolumeClaim:
          claimName: nextcloud-data
---
apiVersion: v1
kind: Service
metadata:
  name: nextcloud
  namespace: nextcloud
spec:
  selector:
    app: nextcloud
  ports:
    - name: http
      port: 80
      targetPort: http
    - name: https
      port: 443
      targetPort: https
```

[!NOTE] - The image runs as root and drops privileges internally via s6 to the `abc` user (uid/gid `PUID`/`PGID`). Do **not** set `runAsUser`; `fsGroup: 1000` makes the PVCs writable. - The admin account is **not** created here. Section 7 installs Nextcloud against PostgreSQL exactly like the Podman record did (via `occ maintenance:install`). Do **not** set the `PASSWORD` environment variable, otherwise the image pre-installs Nextcloud with SQLite and the PostgreSQL install below fails. - `hostAliases` maps `nextcloud.local` to the ingress controller address inside the pod. Adjust the IP to your cluster (see section 9, fix 4).

5.6 `eurooffice.yaml`

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: eurooffice-data
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: eurooffice-logs
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
```

```
  resources:
    requests:
      storage: 10Gi
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: eurooffice-config
  namespace: nextcloud
spec:
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 1Gi
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: eurooffice
  namespace: nextcloud
spec:
  replicas: 1
  selector:
    matchLabels:
      app: eurooffice
  template:
    metadata:
      labels:
        app: eurooffice
    spec:
      # The document server must resolve the Nextcloud hostname server-side
      # to the ingress address (used by the converter/docservice callbacks).
      hostAliases:
        - ip: "192.168.0.114"   # CHANGE ME: ingress controller address
          hostnames:
            - "nextcloud.local"
      containers:
        - name: eurooffice
          image: ghcr.io/euro-office/documentserver:latest
          env:
            - name: JWT_ENABLED
              value: "true"
            - name: JWT_SECRET
              valueFrom:
                secretKeyRef:
                  name: nextcloud-secrets
                  key: JWT_SECRET
            # Fix from nextcloud-euro-office-integration.md: the document
            # server calls back into Nextcloud over https with a self-signed
            # certificate, so TLS verification is disabled (rejectUnauthorized
            # false) and private IPs are accepted as callback targets.
            - name: USE_UNAUTHORIZED_STORAGE
              value: "true"
            - name: ALLOW_PRIVATE_IP_ADDRESS
              value: "true"
          ports:
            - name: http
              containerPort: 80
          volumeMounts:
            - name: data
              mountPath: /var/lib/euro-office/documentserver
            - name: logs
              mountPath: /var/log/euro-office/documentserver
            - name: config
              mountPath: /etc/euro-office/documentserver
          readinessProbe:
            httpGet:
              path: /healthcheck
              port: http
            initialDelaySeconds: 30
```

```
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /healthcheck
              port: http
            initialDelaySeconds: 60
            periodSeconds: 30
          resources:
            requests:
              memory: 2Gi
              cpu: "1"
            limits:
              memory: 8Gi
              cpu: "4"
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: eurooffice-data
        - name: logs
          persistentVolumeClaim:
            claimName: eurooffice-logs
        - name: config
          persistentVolumeClaim:
            claimName: eurooffice-config
---
apiVersion: v1
kind: Service
metadata:
  name: eurooffice
  namespace: nextcloud
spec:
  selector:
    app: eurooffice
  ports:
    - port: 80
      targetPort: http
```

[!NOTE] The two environment variables `USE_UNAUTHORIZED_STORAGE=true` and `ALLOW_PRIVATE_IP_ADDRESS=true` are the **document-saving fix** recorded in nextcloud-euro-office-integration.md. Under Podman they had to be patched into `containers.json` and re-applied after every mastercontainer recreate; in Kubernetes they are plain manifest fields and survive every pod recreation automatically.

Security note: disabling TLS verification is acceptable only for this local-only setup with a self-signed certificate. If the instance moves to a public domain with a real certificate, remove both variables again.

5.7 ingress.yaml

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nextcloud
  namespace: nextcloud
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "10g"
    nginx.ingress.kubernetes.io/proxy-read-timeout: "3600"
    nginx.ingress.kubernetes.io/proxy-send-timeout: "3600"
spec:
  tls:
    - hosts:
        - nextcloud.local
      secretName: nextcloud-tls
  rules:
    - host: nextcloud.local
      http:
        paths:
          - path: /eurooffice
            pathType: Prefix
```

```
      backend:
        service:
          name: eurooffice
          port:
            number: 80
    - path: /
      pathType: Prefix
      backend:
        service:
          name: nextcloud
          port:
            number: 80
```

The TLS secret `nextcloud-tls` is a **self-signed** certificate (the Kubernetes equivalent of the Caddy internal CA from the Podman fixes). Generate it before applying the Ingress:

```
openssl req -x509 -nodes -newkey rsa:2048 -days 3650 \
  -keyout nextcloud.local.key -out nextcloud.local.crt \
  -subj "/CN=nextcloud.local" \
  -addext "subjectAltName=DNS:nextcloud.local,DNS:*.nextcloud.local"

kubectl create secret tls nextcloud-tls \
  --namespace nextcloud \
  --key nextcloud.local.key \
  --cert nextcloud.local.crt
```

> [!NOTE] The self-signed certificate means every browser shows the usual "connection not private" warning, exactly like under Podman. For a trusted certificate use cert-manager with a `ClusterIssuer` of type `SelfSigned` (or an internal CA). If you later switch to a public domain with a real Let's Encrypt certificate, remove the `USE_UNAUTHORIZED_STORAGE` / `ALLOW_PRIVATE_IP_ADDRESS` variables from section 5.6 again.

6. Deploying the stack

Save each manifest from section 5 as a file (e.g. in a directory `kubernetes/`), then apply them in order:

```
kubectl apply -f namespace.yaml
kubectl apply -f secret.yaml
kubectl apply -f postgres.yaml
kubectl apply -f redis.yaml
kubectl apply -f nextcloud.yaml
kubectl apply -f eurooffice.yaml
kubectl apply -f ingress.yaml
```

Wait until all pods are running and ready:

```
kubectl -n nextcloud get pods -w
#  NAME                  READY   STATUS    RESTARTS   AGE
#  eurooffice-xxxxx      1/1     Running   0          2m
#  nextcloud-xxxxx       1/1     Running   0          2m
#  nextcloud-postgres-0  1/1     Running   0          3m
#  nextcloud-redis-xxxx  1/1     Running   0          3m

kubectl -n nextcloud get svc,ingress,pvc
```

7. Nextcloud initial setup (occ)

Nextcloud is installed against PostgreSQL with `occ`, exactly as recorded in final-install.md (`step 6`). The image's web root is `/app/www/public` and `occ` lives at `/app/www/public/occ`.

```
# Install Nextcloud against the PostgreSQL service (first boot)
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ maintenance:install \
  --database=pgsql \
  --database-host=nextcloud-postgres \
  --database-name=nextcloud \
```

```
  --database-user=nextcloud \
  --database-pass=nextcloud-secret \
  --admin-user=admin \
  --admin-pass=nextcloud-admin-pw
```

Configure trusted domains and the reverse-proxy / TLS settings (the Kubernetes equivalent of the AIO domain + overwrite settings):

```
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set trusted_domains 0 --value=localhost
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set trusted_domains 1 --value=nextcloud.local
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set overwriteprotocol --value=https
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set overwritehost --value=nextcloud.local
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set overwritewebroot --value=/

# Trust the Ingress controller so X-Forwarded-* headers are accepted
# (use the CIDR of your ingress controller, e.g. the pod/Service CIDR)
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set trusted_proxies 0 --value=10.0.0.0/8
```

Enable Redis as cache and lock backend (the Kubernetes equivalent of the AIO `nextcloud-aio-redis`):

```
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set redis host --value=nextcloud-redis
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set redis port --value=6379
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set redis password --value=nextcloud-redis-secret
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set memcache.local --value='\OC\Memcache\Redis'
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set memcache.distributed --value='\OC\Memcache\Redis'
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:system:set memcache.locking --value='\OC\Memcache\Redis'
```

> [!NOTE] The values `nextcloud-secret`, `nextcloud-admin-pw`, `nextcloud-redis-secret` match the Secret in section 5.2 and the Podman record. Change them consistently if you use different values.

8. Euro-Office integration

8.1 Install the Euro-Office Nextcloud app

Under Podman the `eurooffice` app was installed by cloning `https://github.com/Euro-Office/eurooffice-nextcloud`, building the frontend and installing Composer dependencies (see final-install.md steps 12-13 and euro-office-github.md). In Kubernetes, packages added inside a running container are **lost on pod recreation**, so the app must be baked into the image.

Build a custom image from the Nextcloud image with the app and the required PHP extensions:

```
FROM node:20 AS eurooffice-build
WORKDIR /build
RUN git clone --recursive https://github.com/Euro-Office/eurooffice-nextcloud.git eurooffice
WORKDIR /build/eurooffice
RUN npm install && npm run build

FROM quay.io/linuxserver.io/nextcloud:latest
RUN apk add --no-cache \
      php83-pdo php83-pdo_pgsql php83-pgsql php83-posix \
      php83-simplexml php83-ctype php83-curl php83-dom php83-gd php83-gmp \
      php83-intl php83-json php83-mbstring php83-openssl php83-session \
      php83-xml php83-xmlreader php83-xmlwriter php83-zip \
      git composer
COPY --from=eurooffice-build /build/eurooffice /app/www/public/apps/eurooffice
WORKDIR /app/www/public/apps/eurooffice
RUN composer install --no-dev --no-interaction
```

```
docker build -t <your-registry>/nextcloud-eurooffice:latest .
docker push <your-registry>/nextcloud-eurooffice:latest
```

Then point the `nextcloud.yaml` deployment at this image (`image: <your-registry>/nextcloud-eurooffice:latest`) and re-apply it:

```
kubectl apply -f nextcloud.yaml
kubectl -n nextcloud rollout restart deploy/nextcloud
kubectl -n nextcloud rollout status deploy/nextcloud
```

> [!NOTE] Building a PHP image in an air-gapped environment requires the Composer/npm packages to be available in a local mirror. An alternative that does not require a custom image is to install the app once via `kubectl exec` (git clone + npm + composer inside the container) - but the result is lost on pod recreation, so this is only acceptable for short-lived tests.

8.2 Enable and configure the app

```
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ app:enable eurooffice

kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:app:set eurooffice DocumentServerUrl --value="https://nextcloud.local/eurooffice/"

kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ config:app:set eurooffice jwt_secret --value="euro-office-jwt-secret-at-least-32-chars"
```

The `jwt_secret` must match the `JWT_SECRET` of the document server (section 5.2 and 5.6).

9. How the Podman fixes translate to Kubernetes

All fixes from the Podman installation are either **obsolete** (because Kubernetes provides the feature) or **encoded in the manifests** (so no runtime patching is ever needed again):

#	Podman problem	Podman fix	Kubernetes equivalent
1	Non-root containers cannot bind host port 443 (`CAP_NET_BIND_SERVICE`)	Patch `containers.json`	**Not needed.** The Ingress controller owns ports 80/443; workloads bind only container ports.
2	Caddy cannot get a Let's Encrypt cert for a `.local` domain (`SSL_ERROR_INTERNAL_ERROR_ALERT`)	Caddy `tls { issuer internal }` (self-signed)	**Ingress + self-signed TLS secret** (or cert-manager), section 5.7.
3	php-fpm `listen.allowed_clients` drops the new apache IP after recreation (HTTP 503)	`podman restart nextcloud-aio-nextcloud`	**Not applicable.** nginx and PHP run in the same pod (localhost), so there is no cross-pod `allowed_clients` to go stale.
4	Euro-Office cannot save: `UNABLE_TO_GET_ISSUER_CERT_LOCALLY` on the callback to the self-signed `https://nextcloud.local`	`USE_UNAUTHORIZED_STORAGE=true` + `ALLOW_PRIVATE_IP_ADDRESS=true` in `local.json` / `containers.json`	**Env vars in the eurooffice Deployment**, section 5.6. Survive every recreate.
5	Docker API version mismatch (`v1.44` vs `v1.41`)	`DOCKER_API_VERSION=1.41`	**Not applicable.** No Docker-compatible API is used; the cluster API is the orchestrator.

#	Podman problem	Podman fix	Kubernetes equivalent
6	Podman socket permissions (`0600` vs `0660`)	`systemctl --user restart podman.socket`	**Not applicable.** No container engine socket inside the cluster.
7	`containers.json` patches lost on mastercontainer recreate	Re-run helper script / systemd unit	**Not applicable.** Manifests are the source of truth; nothing is patched at runtime.

The one *new* requirement on Kubernetes is name resolution for `nextcloud.local` **inside the cluster**: the document server's background services (converter, docservice) call back into Nextcloud over `https://nextcloud.local`, and the pod must resolve that name to the ingress controller address. Two options:

1. `hostAliases` **(used in the manifests above)** - a per-pod `/etc/hosts` entry pointing `nextcloud.local` at the ingress address. Simple and local to the workload, but the IP must match your ingress controller.
2. **Cluster DNS (CoreDNS)** - a hosts block in the CoreDNS Corefile resolves `nextcloud.local` cluster-wide:

```
.:53 {
    ...
    hosts {
        192.168.0.114 nextcloud.local
        fallthrough
    }
}
```

```
kubectl -n kube-system get configmap coredns -o yaml > coredns-cm.yaml
# edit the Corefile (add the hosts block shown above), then:
kubectl apply -f coredns-cm.yaml
kubectl -n kube-system rollout restart deployment/coredns
```

Find the ingress address with `kubectl get svc -A` (the LoadBalancer/NodePort of your ingress controller). On a bare-metal/air-gapped cluster this is usually a node IP.

10. Verification

```
# 1. all workloads ready
kubectl -n nextcloud get pods,svc,ingress,pvc

# 2. document server healthcheck
kubectl exec -n nextcloud deploy/eurooffice -- \
  curl -s http://localhost/healthcheck

# 3. Nextcloud reachable through the ingress
curl -sk -o /dev/null -w '%{http_code}\n' https://nextcloud.local/login   # expect 200

# 4. Euro-Office connection check
kubectl exec -n nextcloud deploy/nextcloud -- \
  php /app/www/public/occ eurooffice:documentserver --check
#  expect: Document server https://nextcloud.local/eurooffice/ ... is successfully connected

# 5. final check: open a document in Euro-Office from Nextcloud and save it
```

11. Management and updates

```
# status
kubectl -n nextcloud get all

# logs
kubectl -n nextcloud logs deploy/nextcloud
kubectl -n nextcloud logs deploy/eurooffice

# restart a workload
kubectl -n nextcloud rollout restart deploy/nextcloud
```

```
kubectl -n nextcloud rollout status deploy/nextcloud

# update an image tag, then:
kubectl -n nextcloud set image deploy/nextcloud nextcloud=<new-image>
kubectl -n nextcloud rollout status deploy/nextcloud

# teardown (data in the PVCs is NOT deleted by deleting pods)
kubectl delete -f ingress.yaml -f eurooffice.yaml -f nextcloud.yaml -f redis.yaml -f postgres.yaml -f secret.yaml -
    f namespace.yaml
```

Because the workloads are managed Deployments/StatefulSets, they restart automatically on node failure, survive pod recreation (including updates), and no "re-run the fix script after recreation" step exists - unlike the Podman installation.

12. Estimated time and risk

Activity	Estimated time
Preparation (images in registry, ingress controller, storage class, DNS)	0.5 - 2 hours
Applying manifests and basic setup (sections 5-7)	0.5 - 1 hour
Euro-Office app image build and integration (section 8)	1 - 3 hours
TLS, DNS/resolution and document-saving verification (sections 9-10)	1 - 2 hours
Total for an experienced admin on an existing cluster	**about 3 - 8 hours**

> [!WARNING] These figures are **estimates produced by artificial intelligence** and are based on the recorded Podman installation, **not** on a real Kubernetes validation. A real deployment on Kubernetes can easily take **considerably more time**: image builds and registry synchronization, ingress/TLS/DNS configuration, storage and permissions problems, Nextcloud app compatibility, and the Euro-Office document-saving integration all regularly require substantial rework in practice. Budget accordingly.

13. Troubleshooting

- **Ingress returns 502/503 ("bad gateway")**: check that the `nextcloud` and `eurooffice` Services select the right pods (`kubectl -n nextcloud get endpoints`). If the backend is HTTPS on port 443, use the `http` backend on port 80 as in the manifests (TLS terminates at the Ingress).
- **Browser shows "connection not private"**: expected - the certificate is self-signed. Install a trusted internal CA (cert-manager) if this must disappear.
- **"The document cannot be saved" in Euro-Office**: the document server cannot call back into Nextcloud. Check (a) `USE_UNAUTHORIZED_STORAGE=true` and `ALLOW_PRIVATE_IP_ADDRESS=true` in the eurooffice Deployment, (b) that `nextcloud.local` resolves inside the pods (`kubectl exec deploy/eurooffice -- getent hosts nextcloud.local`), and (c) the converter log as described in nextcloud-euro-office-integration.md.
- **Nextcloud returns 400 / "not trusted domain"**: complete the `trusted_domains` and `trusted_proxies` settings from section 7.
- **Persistent volumes are not writable**: adjust the `fsGroup` in the pod `securityContext` (section 5.5) to match the image's user. Postgres (uid 999) and the linuxserver image (uid 1000) each need their own group if you see permission errors.
- **Air-gapped cluster cannot pull images**: import the images into your local registry (`docker pull`/`podman pull` on a connected host, then push into the cluster's registry or use `ctr images import` on the node) and adapt the `image:` fields in the manifests.
- **Euro-Office app not enabled after pod restart**: the app was installed at runtime instead of being baked into the image - rebuild the custom image (section 8.1).

14. References

- nextcloud/all-in-one
- Euro-Office/DocumentServer
- Euro-Office/eurooffice-nextcloud
- linuxserver/nextcloud image documentation
- ingress-nginx
- cert-manager